BLOCKCHAIN TECHNOLOGY

Practical Manual

Subject Code: 410243

All 5 practicals with theory, procedures, Solidity programs, deployment steps and viva points.

Practical Index

No.	Practical
1	Install MetaMask and study Ether spending per transaction
2	Create a MetaMask wallet for cryptocurrency transactions
3	Bank Account smart contract: Deposit, Withdraw, Show Balance
4	Student Data using Structures, Arrays and Fallback; observe gas
5	Survey report on Blockchain types and real-time use cases

Practical 1 — MetaMask Installation and Ether Transaction Cost

Aim: Install MetaMask and study the Ether/gas cost of a blockchain transaction.

Theory: MetaMask is a cryptocurrency wallet used to manage blockchain accounts and sign transactions. Ethereum transactions require gas. The transaction fee depends mainly on gas used and gas price.

Procedure:

1. Install the MetaMask browser extension.
2. Create a wallet and securely store the Secret Recovery Phrase offline.
3. Select an Ethereum or test network.
4. Perform a small test transaction.
5. Open transaction details and observe gas used, gas price and transaction fee.

Result: MetaMask was installed and the transaction gas/fee details were studied.

Practical 2 — Create a MetaMask Wallet

Aim: Create a cryptocurrency wallet using MetaMask and understand its use in crypto transactions.

Procedure:

1. Install MetaMask.
2. Select Create a new wallet.
3. Set a strong password.
4. Securely record the Secret Recovery Phrase.
5. Verify the recovery phrase.
6. Note the wallet address used for receiving cryptocurrency.
7. For academic experiments, use a test network and test tokens.

Security: Never share the Secret Recovery Phrase or private key.

Result: A MetaMask wallet was created successfully and its basic transaction workflow was studied.

Practical 3 — Bank Account Smart Contract

Aim: Write and deploy a Solidity smart contract supporting deposit, withdrawal and balance checking.

Solidity Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract BankAccount {
    mapping(address => uint256) private balances;

    function deposit() public payable {
        require(msg.value > 0, "Deposit must be greater than zero");
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint256 amount) public {
        require(amount > 0, "Amount must be greater than zero");
        require(balances[msg.sender] >= amount, "Insufficient balance");
        balances[msg.sender] -= amount;
        payable(msg.sender).transfer(amount);
    }

    function showBalance() public view returns (uint256) {
        return balances[msg.sender];
    }
}
```

Deployment in Remix: Create BankAccount.sol → compile with Solidity 0.8.x → Deploy & Run → select MetaMask/test environment → Deploy → call deposit with an Ether value → call showBalance → call withdraw with an amount.

Result: The Bank Account smart contract was deployed and deposit, withdrawal and balance operations were tested.

Practical 4 — Student Data using Structure, Array and Fallback

Aim: Store student information using Solidity structures and arrays, demonstrate fallback/receive behavior, deploy the contract and observe gas values.

Solidity Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract StudentData {
    struct Student {
        uint id;
        string name;
        uint age;
    }

    Student[] public students;

    event FallbackCalled(address sender, uint value);

    function addStudent(uint _id, string memory _name, uint _age) public {
        students.push(Student(_id, _name, _age));
    }

    function getStudent(uint index)
        public view returns (uint, string memory, uint)
    {
        Student memory s = students[index];
        return (s.id, s.name, s.age);
    }

    fallback() external payable {
        emit FallbackCalled(msg.sender, msg.value);
    }

    receive() external payable {
        emit FallbackCalled(msg.sender, msg.value);
    }
}
```

Concepts: Structure stores id/name/age. Array stores multiple students. Fallback executes when no function matches the call. receive handles plain Ether transfers. State-changing functions consume gas.

Deployment: Compile in Remix → deploy through the selected Ethereum/test network → add student records → retrieve records → inspect MetaMask transaction details for gas used, gas price and fee.

Result: Student data was stored and retrieved, fallback/receive behavior was demonstrated, and transaction gas information was observed.

Practical 5 — Types of Blockchains and Real-Time Use Cases

Aim: Study different types of blockchains and their real-time applications.

1. Public Blockchain: Open participation and generally transparent. Examples include Bitcoin and Ethereum. Uses: cryptocurrency, DeFi and NFTs.

2. Private Blockchain: Controlled by one organization. Uses: internal enterprise records, audit trails and controlled data sharing.

3. Consortium/Federated Blockchain: Governed by a group of organizations. Uses: banking, trade finance and supply-chain networks.

4. Hybrid Blockchain: Combines permissioned/private control with selected public verification. Uses: government, enterprise, identity and supply-chain applications.

Comparison:

Type	Control	Typical Uses
Public	Open/decentralized	Crypto, DeFi, NFTs
Private	Single organization	Enterprise records
Consortium	Multiple organizations	Banking, supply chain
Hybrid	Mixed	Government, enterprise

Conclusion: Blockchain type should be selected according to requirements for decentralization, privacy, governance, scalability and transparency.

Quick Viva Questions

1. What is blockchain? A distributed ledger that records transactions in linked blocks.

2. What is a smart contract? A program stored and executed on a blockchain.

3. What is gas? A unit representing computational work required for an Ethereum operation.

4. What is Solidity? A programming language commonly used to write Ethereum smart contracts.

5. What is MetaMask? A wallet and interface for managing blockchain accounts and signing transactions.

6. Why use test networks? To practice blockchain transactions without risking real cryptocurrency.